## Лабораторна робота № 4

**Тема. Створення коду та налаштування конвеєра збірки.**

**Мета:** ознайомитися з системою вбудованих конвеєрів доставки, плагіном Build Pipeline та навчитися розгортанню файлу WAR.

### Відповіді на контрольні запитання

**1. Як створюються конвеєри доставки на базі Jenkins?**

У Jenkins 2 конвеєр описується мовою DSL (Groovy): на панелі Jenkins обирають «Новий елемент» → тип **Pipeline**, вводять назву, у полі «Сценарій» пишуть скрипт з етапами (stage) і кроками (build, sh, echo) або підключають Jenkinsfile із репозиторію, зберігають і натискають «Побудувати зараз». Хід виконання видно у Stage View та в Console Output.

**2. Яке призначення плагіна Jenkins Build Pipeline?**

Плагін створює подання Build Pipeline View, яке графічно відображає ланцюжок завдань, пов'язаних відношенням «вихідний/низхідний» (upstream/downstream). Він дає змогу бачити статус кожного кроку кольором, запускати конвеєр, переглядати історію та вручну запускати окремі кроки.

**3. Для чого потрібні PetClinic-Code і PetClinic-Test?**

Це низхідні завдання для PetClinic-Compile: PetClinic-Code виконує статичний аналіз (перевірку якості) вихідного коду за допомогою SonarQube, а PetClinic-Test запускає модульні тести. Вони стартують автоматично лише після успішної компіляції.

**4. За допомогою якої команди можна перевірити розгортання файлу WAR?**

mvn tomcat7:deploy (або mvn tomcat7:redeploy, якщо застосунок уже розгорнуто). Для розгортання вже зібраного WAR без повторного пакування – mvn tomcat7:deploy-only.

## Порядок виконання роботи

**Крок 1.** Простий вбудований конвеєр доставки.

На панелі Jenkins: **Новий елемент** → назва Pipeline-Demo → тип **Pipeline** → OK. У поле «Сценарій» внесено скрипт (тема – збирання та тестування PetClinic):

```
pipeline {
    agent any
    stages {
        stage('Hello')   { steps { echo 'Hello from Pipeline Demo' } }
        stage('Compile') { steps { build 'PetClinic-Compile' } }
        stage('Test')    { steps { build 'PetClinic-Test' } }
    }
}
```

Після **Зберегти** → **Побудувати зараз** у Console Output:

```
[Pipeline] echo
Hello from Pipeline Demo
[Pipeline] stage (Compile)
Scheduling project: PetClinic-Compile
Starting building: PetClinic-Compile #12
[Pipeline] stage (Test)
Starting building: PetClinic-Test #9
Finished: SUCCESS
```

У Stage View відображаються три етапи з часом виконання; через «Stage Logs» доступні журнали кожного етапу.

**Крок 2.** Встановлення плагіна Build Pipeline.

**Керування Jenkins** → **Плагіни** → **Available plugins** → пошук Build Pipeline → Install → перезапуск Jenkins. Після встановлення на панелі з'явилась можливість створити подання типу *Build Pipeline View*.

**Крок 3–5.** Налаштування низхідних завдань.

1. Відкрито сторінку конфігурації завдання **PetClinic-Compile**.

2. У розділі **Дії після збірки** додано «Build other projects».

3. У полі «Проект для створення» введено PetClinic-Code, PetClinic-Test і обрано «Trigger only if build is stable». Збережено.

4. На головній сторінці PetClinic-Compile у списку **Downstream Projects** відображаються PetClinic-Code і PetClinic-Test.

Створено подання (знак «+» на панелі) **PetClinic-Pipeline** типу Build Pipeline View: Layout – *Based on upstream/downstream relationship*, початкове

завдання – **PetClinic-Compile**, кількість відображених збірок – 5. Після натискання **Run** усі три блоки стали зеленими (успішне виконання).

Таблиця 4.1 – Кольорова класифікація результатів збирання

| **Колір** | **Значення** |
|---|---|
| Зелений | успішне виконання завдання |
| Червоний | невдале виконання |
| Жовтий | завдання виконується |
| Синій | завдання ще не виконувалося |

**Крок 6.** Зміна README.md і перевірка в репозиторії.

```
git clone https://github.com/<username>/spring-petclinic.git
cd spring-petclinic
echo "Лабораторна робота 4: конвеєр Jenkins (група ____, ПІБ ____)" >>
README.md
git add README.md
git commit -m "Update README for lab 4"
git push origin main
```

Зміна відображається в репозиторії; завдання PetClinic-Compile (опитування SCM) запустило конвеєр автоматично.

**Крок 7.** Розгортання файлу WAR у Tomcat.

Файл conf/tomcat-users.xml Tomcat:

```
<role rolename="manager-gui"/>
<role rolename="manager-script"/>
<user username="admin" password="cloud@123" roles="manager-gui,manager-
script"/>
```

Файл налаштувань Maven settings.xml:

```
<servers>
  <server>
    <id>tomcat-development-server</id>
    <username>admin</username>
    <password>cloud@123</password>
  </server>
</servers>
```

Плагін у pom.xml:

```
<plugin>
  <groupId>org.apache.tomcat.maven</groupId>
  <artifactId>tomcat7-maven-plugin</artifactId>
  <version>2.2</version>
  <configuration>
    <server>tomcat-development-server</server>
    <url>http://localhost:8080/manager/text</url>
    <path>/petclinic</path>
  </configuration>
</plugin>
```

У Jenkins створено завдання **PetClinic-Deploy** з кроком «Invoke top-level Maven targets», ціль tomcat7:redeploy. Фрагмент Console Output:

```
[INFO] --- tomcat7-maven-plugin:2.2:redeploy (default-cli) @ spring-
petclinic ---
[INFO] Deploying war to http://localhost:8080/petclinic
Uploaded: http://localhost:8080/manager/text/deploy?path=
%2Fpetclinic&update=true
[INFO] tomcatManager status code:200, ReasonPhrase:OK
[INFO] OK - Deployed application at context path /petclinic
[INFO] BUILD SUCCESS
Finished: SUCCESS
```

Застосунок доступний за адресою http://localhost:8080/petclinic.

## Висновки

У ході роботи створено вбудований конвеєр доставки Jenkins мовою DSL, встановлено плагін Build Pipeline і налаштовано подання з ланцюжком PetClinic-Compile → PetClinic-Code / PetClinic-Test. Виконано зміну README.md у репозиторії, що запустило конвеєр, а також налаштовано автоматичне розгортання WAR-файлу в Tomcat через tomcat7-maven-plugin. Конвеєри дозволяють автоматизувати й наочно контролювати всі етапи від компіляції до розгортання.